

Ha pensado durante 1m 19s >

Sí. Bajo la representación estándar de grupos finitos mediante sus **tablas de multiplicación**, se puede demostrar algo incluso más fuerte:

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^2]$$

y, por tanto, inmediatamente,

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^4] .$$

Esta idea corresponde a la noción de **nondeterminismo acotado**: $NP[f(n)]$ contiene los problemas resolubles en tiempo polinomial usando solamente $O(f(n))$ bits/pasos nondeterministas.  Bonndoc Para Group Isomorphism, de hecho se conoce la cota de $O(\log^2 n)$ bits nondeterministas cuando n es el orden del grupo.  ECCC +1

1. ¿Qué es GROUP-ISOMORPHISM?

Tenemos dos grupos finitos

$$G = (G, \cdot_G), \quad H = (H, \cdot_H)$$

dados mediante sus tablas de multiplicación.

Queremos decidir si

$$G \cong H.$$

Esto significa determinar si existe una función

$$\varphi : G \rightarrow H$$

que sea una **biyección** y preserve la operación:

$$\varphi(x \cdot_G y) = \varphi(x) \cdot_H \varphi(y)$$

para todo $x, y \in G$.

Esa función φ es un **isomorfismo de grupos**.

Si ambos grupos tienen m elementos, sus tablas tienen esta forma:

$$\begin{array}{cccccc} \cdot_G & g_1 & g_2 & \cdots & g_m \\ g_1 & & & & \\ g_2 & & & & \\ \vdots & & & & \\ g_m & & & & \end{array}$$

Cada tabla contiene m^2 entradas.

2. ¿Qué significa $NP[(\log n)^4]$?

La clase

$$NP[f(n)]$$

es una versión de NP donde limitamos cuánto **nondeterminismo** puede utilizar la máquina.

Formalmente,

$$L \in NP[f(n)]$$

si existe una máquina de Turing nondeterminista M tal que:

$$M$$

trabaja en tiempo polinomial y utiliza solamente

$$O(f(n))$$

bits nondeterministas.

Por tanto,

$$NP[(\log n)^4]$$

permite solamente

$$O((\log n)^4)$$

bits de elecciones nondeterministas.

Esto es muchísimo menor que el número polinomial de bits que permite NP en general.

3. La idea ingenua no sirve

Una primera idea sería:

Adivinemos completamente el isomorfismo $\varphi : G \rightarrow H$.

Pero eso significa adivinar

$$\varphi(g_1), \varphi(g_2), \dots, \varphi(g_m).$$

Cada elemento de H necesita aproximadamente

$$\log m$$

bits para ser identificado.

Por tanto necesitaríamos

$$m \log m$$

bits nondeterministas.

Esto es demasiado grande:

$$m \log m \notin O((\log m)^4).$$

La clave consiste en observar que **no necesitamos adivinar toda la función**.

4. Lema fundamental: pocos elementos generan cualquier grupo finito

Sea G un grupo con

$$|G| = m.$$

Entonces existe un conjunto generador

$$S = \{g_1, \dots, g_k\}$$

tal que

$$k \leq \lceil \log_2 m \rceil.$$

Veamos por qué.

Comenzamos con el subgrupo trivial

$$G_0 = \{e\}.$$

Si

$$G_0 \neq G,$$

escogemos

$$g_1 \notin G_0$$

y construimos

$$G_1 = \langle g_1 \rangle.$$

Si todavía

$$G_1 \neq G,$$

escogemos

$$g_2 \notin G_1$$

y hacemos

$$G_2 = \langle g_1, g_2 \rangle.$$

Continuamos así:

$$G_i = \langle g_1, \dots, g_i \rangle.$$

Como

$$g_{i+1} \notin G_i,$$

tenemos una inclusión propia

$$G_i < G_{i+1}.$$

Por el teorema de Lagrange,

$$[G_{i+1} : G_i] \geq 2,$$

por lo que

$$|G_{i+1}| \geq 2|G_i|.$$

Así,

$$|G_0| = 1,$$

$$|G_1| \geq 2,$$

$$|G_2| \geq 2^2,$$

y en general

$$|G_k| \geq 2^k.$$

Pero $G_k \subseteq G$, de modo que

$$2^k \leq m.$$

Tomando logaritmos:

$$k \leq \log_2 m .$$

Este es el resultado esencial.

5. ¿Por qué esto ayuda?

Supongamos que

$$G = \langle g_1, \dots, g_k \rangle$$

con

$$k \leq \log_2 m.$$

Si existe un isomorfismo

$$\varphi : G \rightarrow H,$$

entonces conocer únicamente

$$\varphi(g_1), \dots, \varphi(g_k)$$

determina completamente a φ .

¿Por qué?

Porque todos los elementos de G pueden escribirse usando los generadores. Por ejemplo,

$$x = g_1 g_3 g_1 g_2.$$

Entonces necesariamente

$$\varphi(x) = \varphi(g_1)\varphi(g_3)\varphi(g_1)\varphi(g_2).$$

Así que no necesitamos adivinar las imágenes de los m elementos.

Solamente las imágenes de

$$O(\log m)$$

generadores.

6. Cantidad de bits nondeterministas

Tenemos como máximo

$$k \leq \log_2 m$$

generadores.

Para cada generador g_i , debemos adivinar un elemento

$$h_i \in H.$$

Hay m posibilidades.

Escoger una de m posibilidades necesita

$$\lceil \log_2 m \rceil$$

bits.

Por tanto, para todos los generadores utilizamos

$$k \log m$$

bits.

Como

$$k \leq \log m,$$

obtenemos

$$k \log m \leq (\log m)^2.$$

Por lo tanto:

$$O((\log m)^2)$$

bits nondeterministas son suficientes.

Incluso podríamos adivinar tanto los generadores g_i como sus imágenes h_i ; eso solamente multiplicaría la cantidad por una constante:

$$2(\log m)^2 = O((\log m)^2).$$

7. El algoritmo nondeterminista

La máquina realiza el siguiente procedimiento:

1. Comprueba que $|G| = |H| = m$. Adivina como máximo $\lceil \log_2 m \rceil$ generadores $g_1, \dots, g_k$ de G y elementos $h_1, \dots, h_k \in H$, interpretando que queremos $\varphi(g_i) = h_i$.

2. Determinísticamente calcula

$$\langle g_1, \dots, g_k \rangle$$

usando la tabla de multiplicación. Si no obtiene todo G , rechaza.

3. A partir de

$$g_i \mapsto h_i,$$

construye una función candidata

$$\varphi : G \rightarrow H.$$

Como los g_i generan todo G , cada elemento puede obtenerse multiplicando generadores, así que su imagen puede calcularse usando las correspondientes multiplicaciones de los h_i .

4. Comprueba determinísticamente que φ sea una biyección y que para todos $x, y \in G$,

$$\varphi(xy) = \varphi(x)\varphi(y).$$

Si todo se cumple, acepta.

Todas estas verificaciones se hacen directamente sobre las tablas de multiplicación y requieren a lo sumo un número polinomial de operaciones, por ejemplo $O(m^2)$ o $O(m^3)$, dependiendo de cómo implementemos la generación del subgrupo.

8. Correctitud

Tenemos que probar las dos direcciones.

Si $G \cong H$, la máquina puede aceptar

Supongamos que existe

$$\psi : G \rightarrow H$$

isomorfismo.

Por el lema anterior existe

$$S = \{g_1, \dots, g_k\}, \quad k \leq \log_2 m$$

que genera G .

La máquina nondeterminista puede adivinar precisamente

$$h_i = \psi(g_i).$$

Entonces al extender esas imágenes obtendrá exactamente ψ .

Como ψ es biyectiva y

$$\psi(xy) = \psi(x)\psi(y),$$

todas las verificaciones tendrán éxito.

Por tanto,

$$G \cong H \implies \text{existe una rama que acepta.}$$

Si la máquina acepta, $G \cong H$

Si una rama acepta, se ha construido una función

$$\varphi : G \rightarrow H$$

tal que es biyectiva y satisface

$$\varphi(xy) = \varphi(x)\varphi(y)$$

para todo $x, y \in G$.

Eso es exactamente la definición de un isomorfismo.

Por tanto,

$$\text{la máquina acepta} \implies G \cong H.$$

La máquina decide correctamente GROUP-ISOMORPHISM.

9. Cuidado con qué significa n

Aquí hay una distinción importante para escribir la demostración formalmente.

Hasta ahora llamé

$$m = |G| = |H|$$

al número de elementos del grupo.

Pero en teoría de complejidad normalmente n representa la **longitud total de la entrada en bits**.

Como una tabla tiene m^2 posiciones y cada posición almacena uno de m elementos, cada entrada necesita

$$O(\log m)$$

bits.

Por tanto, para las dos tablas:

$$n = \Theta(m^2 \log m).$$

En particular,

$$\log m = O(\log n).$$

Entonces nuestra cantidad de bits nondeterministas

$$O((\log m)^2)$$

es también

$$O((\log n)^2).$$

Por consiguiente,

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^2].$$

Y como

$$(\log n)^2 \leq (\log n)^4$$

para n suficientemente grande,

$$NP[(\log n)^2] \subseteq NP[(\log n)^4].$$

Finalmente,

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^4].$$

Este resultado incluso es más fuerte que el solicitado: el argumento clásico de enumeración de generadores coloca Group Isomorphism con tablas de Cayley en nondeterminismo $O(\log^2 n)$.  ECCC +1

Forma corta de la idea

La demostración completa puede condensarse en:

$$|G| = m$$

⇓

G tiene $\leq \log_2 m$ generadores

⇓

adivinamos sus imágenes en H

⇓

$(\log m)(\log m) = O(\log^2 m)$ bits nondeterministas

⇓

verificamos el isomorfismo en tiempo polinomial

⇓

$$\text{GROUP-ISOMORPHISM} \in NP[\log^2 n] \subseteq NP[\log^4 n].$$



Dame un ejemplo para poder entenderlo

Claro. Veámoslo con un ejemplo pequeño donde podamos ejecutar exactamente la idea de la demostración.

Tomemos dos grupos de 4 elementos.

$$G = (\mathbb{Z}_4, +) = \{0, 1, 2, 3\}$$

con suma módulo 4, y

$$H = \{1, i, -1, -i\}$$

con la multiplicación usual de números complejos.

Queremos determinar si

$$G \cong H.$$

1. No adivinamos toda la función

Una forma ingenua sería adivinar:

$$\varphi(0), \quad \varphi(1), \quad \varphi(2), \quad \varphi(3).$$

Es decir, adivinar la imagen de los 4 elementos.

Pero la demostración nos dice que eso no es necesario.

Observa que en $\mathbb{Z}_4$,

$$1$$

por sí solo genera todo el grupo:

$$1,$$

$$1 + 1 = 2,$$

$$1 + 1 + 1 = 3,$$

$$1 + 1 + 1 + 1 = 0.$$

Por tanto,

$$G = \langle 1 \rangle.$$

Así que basta saber **a dónde enviamos al generador 1**.

2. La máquina nondeterminista hace una elección

Supongamos que la máquina adivina:

$$\varphi(1) = i .$$

Eso es todo lo que necesita adivinar.

Ahora las demás imágenes quedan forzadas.

Como

$$2 = 1 + 1,$$

un homomorfismo debe cumplir:

$$\varphi(2) = \varphi(1 + 1) = \varphi(1)\varphi(1).$$

Entonces:

$$\varphi(2) = i^2 = -1.$$

Luego,

$$3 = 1 + 1 + 1,$$

por lo que

$$\varphi(3) = i^3 = -i.$$

Finalmente,

$$0 = 1 + 1 + 1 + 1,$$

entonces

$$\varphi(0) = i^4 = 1.$$

Obtenemos:

$$\varphi(0) = 1$$

$$\varphi(1) = i$$

$$\varphi(2) = -1$$

$$\varphi(3) = -i$$

Fíjate en algo importante:

Solo adivinamos $\varphi(1) = i$. Las otras tres imágenes se calcularon determinísticamente.

3. Verificamos que sea biyectiva

Las imágenes son:

$$1, i, -1, -i.$$

Son todas distintas y contienen todos los elementos de H .

Por tanto,

$$\varphi : G \rightarrow H$$

es biyectiva.

4. Verificamos que preserve la operación

Necesitamos que

$$\varphi(a + b) = \varphi(a)\varphi(b).$$

Por ejemplo, tomemos

$$2 + 3 = 5 \equiv 1 \pmod{4}.$$

Entonces:

$$\varphi(2 + 3) = \varphi(1) = i.$$

Por otro lado:

$$\varphi(2)\varphi(3) = (-1)(-i) = i.$$

Por tanto,

$$\varphi(2 + 3) = \varphi(2)\varphi(3).$$

Otro ejemplo:

$$3 + 3 = 6 \equiv 2 \pmod{4}.$$

Entonces:

$$\varphi(3 + 3) = \varphi(2) = -1.$$

Mientras que

$$\varphi(3)\varphi(3) = (-i)(-i) = -1.$$

También funciona.

La máquina puede verificar sistemáticamente todas las parejas:

$$(a, b) \in G \times G.$$

Como hay $4^2 = 16$ parejas, esto es fácil.

En el caso general habría m^2 parejas, lo cual sigue siendo polinomial.

5. ¿Dónde aparecen los bits nondeterministas?

Aquí:

$$|G| = |H| = 4.$$

Para representar uno de los cuatro elementos de H necesitamos

$$\log_2 4 = 2$$

bits.

Por ejemplo:

Elemento	Código
1	00
i	01
-1	10
$-i$	11

Cuando la máquina adivina

$$\varphi(1) = i,$$

realmente puede verse como que adivina:

$$01.$$

Es decir, solo utilizó

$$2$$

bits nondeterministas.

Ahora recuerda que

$$(\log_2 4)^2 = 2^2 = 4.$$

Así que estamos dentro del límite:

$$2 \leq 4.$$

En un grupo general, podría haber hasta

$$\log m$$

generadores, y cada imagen necesitaría

$$\log m$$

bits.

Por eso:

$$\begin{array}{ccccc} \log m & \times & \log m & = & (\log m)^2 . \\ \text{generadores} & & \text{bits por imagen} & & \end{array}$$

6. ¿Qué pasa si la máquina adivina mal?

Esto también ayuda mucho a entender NP .

Supongamos que adivina:

$$\varphi(1) = -1.$$

Entonces:

$$\varphi(2) = \varphi(1)^2 = (-1)^2 = 1.$$

Y

$$\varphi(3) = (-1)^3 = -1.$$

Obtendríamos:

$$0 \mapsto 1$$

$$1 \mapsto -1$$

$$2 \mapsto 1$$

$$3 \mapsto -1.$$

Pero esto no es una biyección porque, por ejemplo,

$$\varphi(0) = \varphi(2) = 1.$$

Entonces esa rama de la máquina **rechaza**.

Esto ilustra perfectamente el nondeterminismo:

la máquina prueba posibles imágenes de los generadores

y basta con que exista **una elección correcta** para que acepte.

En nuestro caso:

$$1 \mapsto i$$

funciona, así que

$$\mathbb{Z}_4 \cong \{1, i, -1, -i\}.$$

La idea visual

Todo el argumento puede imaginarse así:

$$G = \{0, 1, 2, 3\}$$

Sabemos que

1

genera todo:

1

↓

la máquina adivina solamente

$$\varphi(1) = i$$

↓

y automáticamente calcula

$$1 + 1 = 2 \quad \Rightarrow \quad \varphi(2) = i^2 = -1$$

$$1 + 1 + 1 = 3 \quad \Rightarrow \quad \varphi(3) = i^3 = -i$$

$$1 + 1 + 1 + 1 = 0 \quad \Rightarrow \quad \varphi(0) = i^4 = 1$$

Así, de **una pequeña cantidad de información adivinada** reconstruimos el isomorfismo completo.

Ese es exactamente el truco de la demostración de

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^4] .$$

De hecho, el ejemplo permite ver por qué obtenemos la cota más fuerte $NP[(\log n)^2]$.



Dame la teoría de NP, L No determinismo y los demás
conceptos de complejidad computacional que nombraste

Claro. Te organizo la teoría desde la base, porque para entender una afirmación como

$$\text{GROUP-ISOMORPHISM} \in NP[(\log n)^4]$$

conviene distinguir muy bien **tiempo, espacio, determinismo, no determinismo,**

certificados y cantidad de bits nondeterministas.

1. Problema de decisión y lenguaje

En complejidad computacional normalmente estudiamos **problemas de decisión**, es decir, problemas cuya respuesta es:

Sí o No.

Por ejemplo:

Dado un grafo G y un entero k , ¿existe un clique de tamaño k ?

Cada problema de decisión puede representarse mediante un **lenguaje**.

Si Σ es un alfabeto, por ejemplo

$$\Sigma = \{0, 1\},$$

entonces

$$\Sigma^*$$

es el conjunto de todas las cadenas finitas sobre ese alfabeto.

Un lenguaje es simplemente

$$L \subseteq \Sigma^*.$$

La interpretación es:

$$x \in L$$

si la respuesta para la instancia x es **Sí**.

Por ejemplo, SAT puede verse como

$$SAT = \{\varphi : \varphi \text{ es una fórmula booleana satisfacible}\}.$$

2. Tamaño de la entrada n

Cuando escribimos expresiones como

$$O(n), \quad O(n^2), \quad O(\log n),$$

n normalmente significa:

$$n = |x|$$

es decir, la cantidad de bits necesarios para representar la entrada.

Esto es importante porque n no siempre es el número de objetos del problema.

Por ejemplo, si un grupo tiene m elementos y se representa mediante una tabla de multiplicación, la tabla tiene

$$m^2$$

posiciones.

Entonces la longitud real de la entrada es aproximadamente

$$n = \Theta(m^2 \log m).$$

3. Máquina de Turing determinista

Una **máquina de Turing determinista**, DTM, es un modelo matemático de computador.

En cada instante, dependiendo del:

- estado actual;
- símbolo leído;

existe exactamente **una transición posible**.

Es decir, la computación sigue un único camino:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots$$

donde cada C_i es una configuración de la máquina.

Podemos pensarla como un programa normal:

```
si condición:
    hacer A
si no:
    hacer B
```

Una vez conocida la entrada, todo está determinado.

4. Máquina de Turing no determinista

Una máquina de Turing no determinista, NTM, puede tener varias opciones posibles desde una misma configuración.

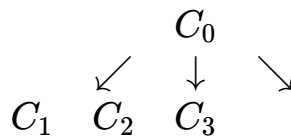
Por ejemplo:

$$C_0$$

puede producir:

$$C_1, C_2, C_3.$$

Entonces la computación forma un árbol:



y cada uno puede volver a ramificarse.

Una NTM **acepta** una entrada si existe al menos una rama que acepta.

Formalmente:

$$x \in L \iff \exists \text{ una rama computacional de } M \text{ que acepta } x.$$

Este

$$\exists$$

es la idea fundamental detrás de NP .

5. ¿Qué significa "adivinar"?

Cuando decimos:

La máquina no determinista adivina una solución.

No significa magia ni aleatoriedad.

Significa que la máquina crea conceptualmente todas las elecciones posibles.

Por ejemplo, para SAT:

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$$

la máquina puede elegir:

$$x_1 \in \{0, 1\},$$

$$x_2 \in \{0, 1\},$$

$$x_3 \in \{0, 1\}.$$

Esto genera 2^3 ramas:

$$000, 001, 010, \dots, 111.$$

Si alguna asignación satisface φ , la máquina acepta.

6. La clase P

La clase

$$P$$

contiene los problemas de decisión que una máquina **determinista** puede resolver en tiempo polinomial.

Formalmente:

$$P = \bigcup_{k \geq 1} DTIME(n^k).$$

Por ejemplo, tiempos como

$$O(n),$$

$$O(n^2),$$

$$O(n^{10})$$

son polinomiales.

En cambio:

$$O(2^n), \quad O(3^n), \quad O(n!)$$

son exponenciales o peores.

Ejemplos conocidos de problemas en P

Encontrar si existe un camino entre dos vértices:

$$PATH \in P.$$

Encontrar el camino mínimo:

$$\text{Shortest Path} \in P.$$

Ordenar números también puede hacerse en tiempo polinomial.

7. La clase NP

NP significa:

Nondeterministic Polynomial Time .

Un lenguaje pertenece a NP si una máquina de Turing **no determinista** puede decidirlo en tiempo polinomial.

Formalmente:

$$NP = \bigcup_{k \geq 1} NTIME(n^k).$$

Pero existe una interpretación mucho más útil.

8. NP mediante certificados y verificadores

Podemos definir NP usando **certificados**.

Un lenguaje L pertenece a NP si existe un algoritmo determinista V , llamado

verificador, tal que:

$$x \in L \iff \exists y : V(x, y) = 1,$$

donde:

- x es la instancia;
- y es un certificado;
- $|y|$ es polinomial en $|x|$;
- V trabaja en tiempo polinomial.

Es decir:

NP = problemas cuyas respuestas Sí poseen certificados verificables eficiente

Ejemplo: SAT

Supongamos:

$$\varphi = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3).$$

El certificado podría ser:

$$y = (x_1 = 1, x_2 = 0, x_3 = 1).$$

El verificador simplemente sustituye los valores y verifica:

$$\varphi(y) = 1.$$

Eso puede hacerse rápidamente.

Por eso:

$$SAT \in NP.$$

9. Relación entre P y NP

Siempre:

$$P \subseteq NP.$$

¿Por qué?

Porque si podemos **resolver** un problema eficientemente, entonces evidentemente podemos verificar una solución eficientemente.

El gran problema abierto es:

$$P \stackrel{?}{=} NP .$$

Todavía no sabemos si:

$$P = NP$$

o

$$P \neq NP.$$

10. NP no significa "no polinomial"

Este error es muy común.

$$NP$$

no significa

No Polinomial.

Significa:

Nondeterministic Polynomial Time .

Además:

$$P \subseteq NP,$$

por lo que muchos problemas en NP también son polinomiales.

11. Bits nondeterministas

Aquí llegamos directamente a la demostración de Group Isomorphism.

Supongamos que una máquina necesita realizar k decisiones binarias:

$$b_1, b_2, \dots, b_k,$$

con

$$b_i \in \{0, 1\}.$$

Decimos que utiliza:

k bits nondeterministas .

Esos k bits determinan una rama de la computación:

$$b_1 b_2 \dots b_k.$$

Como existen:

$$2^k$$

cadenas binarias posibles, existen como máximo 2^k posibles elecciones.

Ejemplo

Si usamos:

$$k = 3$$

bits nondeterministas, tenemos:

$$000, 001, 010, 011, 100, 101, 110, 111.$$

Hay:

$$2^3 = 8$$

posibilidades.

12. ¿Por qué elegir un elemento entre m requiere $\log m$ bits?

Supongamos que debemos elegir uno de:

$$m$$

elementos.

Con k bits podemos representar:

$$2^k$$

posibilidades.

Necesitamos:

$$2^k \geq m.$$

Tomando logaritmo base 2:

$$k \geq \log_2 m.$$

Por tanto necesitamos aproximadamente:

$$\log_2 m$$

bits.

Por ejemplo:

$$m = 8.$$

Entonces:

$$\log_2 8 = 3.$$

Y efectivamente:

$$000, 001, \dots, 111$$

representan los ocho elementos.

13. Nondeterminismo acotado

En NP convencional se permite una cantidad polinomial de bits nondeterministas.

Por ejemplo:

$$n,$$

$$n^2,$$

etc.

Pero podemos estudiar clases mucho más restrictivas.

Una notación usada es:

$$NP[f(n)].$$

Intuitivamente:

$$NP[f(n)]$$

contiene problemas que:

1. se ejecutan en tiempo polinomial;
2. utilizan solamente

$$O(f(n))$$

bits nondeterministas.

Por ejemplo:

$$NP[\log n],$$

$$NP[(\log n)^2],$$

$$NP[(\log n)^4].$$

Claramente:

$$NP[\log n] \subseteq NP[(\log n)^2] \subseteq NP[(\log n)^4] \subseteq NP.$$

14. ¿Qué es $\log n$?

En complejidad normalmente usamos:

$$\log n = \log_2 n.$$

Pero la base no importa para notación asintótica porque:

$$\log_a n = \frac{\log_b n}{\log_b a}.$$

Solo cambia por una constante.

Por ejemplo:

$$\log_2(1024) = 10.$$

Así que aunque la entrada tenga tamaño 1024, una cantidad

$$O(\log n)$$

es aproximadamente 10.

Por eso se dice que el crecimiento logarítmico es muy pequeño.

15. Polylog

Una función de la forma:

$$(\log n)^k$$

para una constante k se denomina **polilogarítmica**.

Por ejemplo:

$$\begin{aligned} &\log n, \\ &(\log n)^2, \\ &(\log n)^4. \end{aligned}$$

Todas son:

$$\text{polylog}(n).$$

Crece muchísimo más lentamente que:

n .

Por ejemplo, si:

$$n = 2^{20} \approx 1\,000\,000,$$

entonces:

$$\log_2 n = 20,$$

mientras que:

$$n \approx 1\,000\,000.$$

16. Espacio computacional

Hasta ahora hablábamos de **tiempo**.

También podemos medir cuánta **memoria** utiliza un algoritmo.

Definimos:

$$SPACE(f(n))$$

como los problemas que pueden resolverse usando:

$$O(f(n))$$

celdas de memoria.

17. La clase L

$$L = SPACE(\log n)$$

se llama **logarithmic space** o espacio logarítmico.

Contiene problemas que una máquina determinista puede resolver usando solamente:

$$O(\log n)$$

memoria adicional.

Es una cantidad extremadamente pequeña de memoria.

Ejemplo intuitivo

Supongamos que queremos recorrer una entrada de n elementos.

Para almacenar la posición actual necesitamos un contador:

$$0, \dots, n - 1.$$

Representar el contador requiere:

$$\log_2 n$$

bits.

Por eso muchos algoritmos que solamente necesitan unos pocos índices pueden trabajar en espacio logarítmico.

18. Algo importante sobre espacio

Cuando decimos:

$$L = SPACE(\log n),$$

normalmente **no contamos la memoria donde está almacenada la entrada.**

La entrada se encuentra en una cinta de solo lectura.

Contamos únicamente el espacio de trabajo adicional.

La estructura puede imaginarse así:

$$\text{Entrada} \longrightarrow \text{máquina} \longleftrightarrow O(\log n) \text{ memoria}$$

19. NL : espacio logarítmico no determinista

La versión no determinista de L se llama:

$$NL = NSPACE(\log n) .$$

Es decir, problemas que una máquina no determinista puede resolver utilizando:

$$O(\log n)$$

espacio.

Un ejemplo fundamental es:

$$PATH$$

o alcanzabilidad dirigida:

Dados vértices s, t , ¿existe un camino desde s hasta t ?

Este problema está en NL .

¿Por qué $PATH$ está en NL ?

La máquina puede hacer:

1. empezar en s ;
2. adivinar un vecino;
3. moverse a ese vecino;
4. repetir hasta llegar a t .

Solo necesita almacenar:

- el vértice actual;
- un contador.

Cada vértice necesita:

$$O(\log n)$$

bits.

Por eso:

$$PATH \in NL.$$

20. Diferencia fundamental entre L y NL

L :

máquina determinista + $O(\log n)$ espacio.

NL :

máquina no determinista + $O(\log n)$ espacio.

Sabemos:

$$L \subseteq NL .$$

Pero no sabemos si:

$$L = NL.$$

21. Tiempo frente a espacio

Debemos separar estos conceptos:

Tiempo

Cuenta cuántos pasos realiza la máquina.

Ejemplo:

$$O(n^2).$$

Espacio

Cuenta cuánta memoria adicional necesita.

Ejemplo:

$$O(\log n).$$

Un algoritmo puede tardar mucho tiempo pero utilizar poca memoria.

22. Relación entre algunas clases importantes

Tenemos:

$$L \subseteq NL \subseteq P \subseteq NP$$

Una forma visual:

$$L \subseteq NL \subseteq P \subseteq NP .$$

No sabemos si algunas de estas inclusiones son estrictas.

En particular:

$$P \stackrel{?}{=} NP$$

y

$$L \stackrel{?}{=} NL$$

son problemas abiertos.

23. NP-completo

Un problema A es **NP-completo** si:

$$A \in NP$$

y además todos los problemas de NP pueden reducirse a A en tiempo polinomial.

Formalmente:

$$\forall L \in NP : L \leq_p A.$$

Por tanto un NP-completo es, informalmente, uno de los problemas más difíciles de NP .

Ejemplos:

SAT ,

$3SAT$,

$CLIQUE$,

VERTEX COVER,
HAMILTONIAN CYCLE.

24. Reducción en tiempo polinomial

Decimos:

$$A \leq_p B$$

si existe una función f , calculable en tiempo polinomial, tal que:

$$x \in A \iff f(x) \in B.$$

La interpretación es:

Si sé resolver B , puedo usarlo para resolver A .

Por ejemplo:

$$SAT \leq_p 3SAT.$$

Esta es precisamente la clase de reducción que has estado estudiando.

25. coNP

Existe otra clase importante:

$$coNP.$$

Si $L \in NP$, entonces su complemento:

$$L$$

pertenece a $coNP$.

Una interpretación:

- NP : existen certificados eficientes para respuestas **Sí**.
- $coNP$: existen certificados eficientes para respuestas **No** del problema original.

Un ejemplo famoso:

$$TAUT = \{\varphi : \varphi \text{ es una tautología}\}$$

está en $coNP$.

No sabemos si:

$$NP = coNP.$$

26. Volvamos a Group Isomorphism

Ahora podemos interpretar completamente:

$$GROUP-ISOMORPHISM \in NP[(\log n)^4].$$

Quiere decir:

Existe una máquina no determinista que decide si dos grupos son isomorfos en tiempo polinomial utilizando como máximo $O((\log n)^4)$ bits de elecciones nondeterministas.

En nuestra demostración obteníamos incluso:

$$O((\log n)^2).$$

¿Por qué?

Si un grupo tiene m elementos, podemos encontrar un conjunto generador de tamaño:

$$O(\log m).$$

La máquina adivina las imágenes de esos generadores.

Cada imagen necesita:

$$O(\log m)$$

bits.

Entonces:

$$O(\log m) \cdot O(\log m) = O((\log m)^2).$$

generadores bits por generador

Después toda la comprobación es determinista y polinomial.

Así:

$$GI \in NP[(\log n)^2].$$

Y como:

$$(\log n)^2 \leq (\log n)^4,$$

tenemos:

$$GI \in NP[(\log n)^4].$$

27. Diferencia importante: L vs $NP[\log n]$

Es fácil confundirlas.

L

Limita la **memoria**:

$$L = SPACE(\log n).$$

La máquina es determinista.

$NP[\log n]$

Limita el **número de bits nondeterministas**:

$$O(\log n).$$

Pero la máquina puede utilizar tiempo y memoria polinomiales para hacer la verificación.

Por tanto:

espacio logarítmico $\neq$ nondeterminismo logarítmico.

Son recursos diferentes.

28. Resumen conceptual

Puedes organizar las clases así:

Clase	Tipo de máquina	Recurso limitado
L	Determinista	$O(\log n)$ espacio
NL	No determinista	$O(\log n)$ espacio
P	Determinista	Tiempo polinomial
NP	No determinista	Tiempo polinomial
$NP[f(n)]$	No determinista	Tiempo polinomial + $O(f(n))$ bits nondeterministas
$coNP$	Relacionada con complementos de NP	Certificados para respuestas negativas

Y las relaciones principales son:

$$L \subseteq NL \subseteq P \subseteq NP$$

mientras que:

$$NP[(\log n)^2] \subseteq NP[(\log n)^4] \subseteq NP.$$

La idea más importante para tu ejercicio es esta distinción:

NP normal permite un certificado polinomial;

mientras que

$NP[(\log n)^4]$ exige que la parte "adivinada" tenga solo $O((\log n)^4)$ bits.

Y eso es justamente lo especial de la demostración de **Group Isomorphism**: no se adivina el isomorfismo completo; se adivina una descripción muy pequeña a partir de generadores.



Ramificado desde **Demostración Group Isomorphism NP**

ChatGPT puede cometer errores. Considera verificar la información importante.